

MDTS4214_711_PS_5

Daniel D. Mondal

2026-02-25

Problem Set - 3 (Continued)

3. Problem to demonstrate the role of qualitative (ordinal) predictors in addition to quantitative predictors in multiple linear regression

Consider “diamonds” data set in R. It is in the ggplot2 package. Make a list of all the ordinal categorical variables. Identify the response.

```
rm(list=ls())
library(ggplot2)
library(stargazer)
library(MASS)

attach(diamonds)
head(diamonds)

## # A tibble: 6 × 10
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.23 Ideal      E      SI2      61.5    55   326   3.95   3.98   2.43
## 2  0.21 Premium    E      SI1      59.8    61   326   3.89   3.84   2.31
## 3  0.23 Good       E      VS1      56.9    65   327   4.05   4.07   2.31
## 4  0.29 Premium    I      VS2      62.4    58   334   4.2    4.23   2.63
## 5  0.31 Good       J      SI2      63.3    58   335   4.34   4.35   2.75
## 6  0.24 Very Good J      VVS2      62.8    57   336   3.94   3.96   2.48

dim(diamonds)

## [1] 53940      10
```

- (a) Run a linear regression of the response on the quality of cut. Write the fitted regression model.

```
lvl=as.numeric(diamonds$cut)
code_fun=function(x){
  if(x-5==0){
    return(c(0,0,0,0))
  }else if(x-5==1){
    return(c(1,0,0,0))
  }else if(x-5==2){
    return(c(1,1,0,0))
  }else if(x-5==3){
    return(c(1,1,1,0))
  }else {
    return(c(1,1,1,1))
  }
}
```

```

    }
  }
  pred=t(sapply(lvl,code_fun))
  colnames(pred)=c("Premium","Very_Good","Good","Fair")
  diamonds=cbind(diamonds,pred)
  ord.model=lm(price~Premium+Very_Good+Good+Fair,data=diamonds)
  summary(ord.model)

##
## Call:
## lm(formula = price ~ Premium + Very_Good + Good + Fair, data = diamonds)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4258  -2741  -1494   1360  15348
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3457.54     27.00  128.051 < 2e-16 ***
## Premium       1126.72     43.22   26.067 < 2e-16 ***
## Very_Good     -602.50     49.39  -12.198 < 2e-16 ***
## Good          -52.90     67.10   -0.788  0.43055
## Fair           429.89    113.85    3.776  0.00016 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3964 on 53935 degrees of freedom
## Multiple R-squared:  0.01286,    Adjusted R-squared:  0.01279
## F-statistic: 175.7 on 4 and 53935 DF,  p-value: < 2.2e-16

```

(b) Test whether the expected price of diamond with premium cut is significantly different from that of the ideal cut.

To Test: $H_0: \beta_{\text{Premium}} - \beta_{\text{Ideal}} = 0$ vs $H_1: \beta_{\text{Premium}} - \beta_{\text{Ideal}} \neq 0$

here($\beta_{\text{Ideal}} = \beta_0$)

the statistic is of the form: $t = \frac{\hat{\beta}_{\text{Premium}}}{SE(\hat{\beta}_{\text{Premium}})}$

The observed p-value for the test is 2^{-16} , which is much smaller than our level of significance (say $\alpha = 0.05$), hence we can say that significant difference exists between the two levels.

(c) What is the expected price of a diamond of ideal cut? The Expected price of the Ideal cut is \$3457.54 on an average.

(d) Modify the regression model in (a) by incorporating the predictor “table”. Write the fitted regression model.

```

ord.model1=lm(price~Premium+Very_Good+Good+Fair+table,data=diamonds)
summary(ord.model1)

```

```
##
## Call:
## lm(formula = price ~ Premium + Very_Good + Good + Fair + table,
##     data = diamonds)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5630  -2694  -1458   1346  15690
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -6563.672    517.450  -12.685 < 2e-16 ***
## Premium      626.220     50.215   12.471 < 2e-16 ***
## Very_Good    -461.015     49.761   -9.265 < 2e-16 ***
## Good         -185.162     67.220   -2.755  0.00588 **
## Fair          365.568    113.504    3.221  0.00128 **
## table         179.105      9.236   19.393 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3950 on 53934 degrees of freedom
## Multiple R-squared:  0.0197, Adjusted R-squared:  0.01961
## F-statistic: 216.7 on 5 and 53934 DF,  p-value: < 2.2e-16
```

(e) Test for the significance of “table” in predicting the price of diamond.

Here we test we apply the students t-test:

To test: $H_0: \beta_{table} = 0$ vs $H_1: \beta_{table} \neq 0$

the statistic under H_0 : $t = \frac{\hat{\beta}_{table}}{SE(\beta_{table})}$

The observed p-value for the test is 2^{-16} , which is much smaller than our level of significance (say $\alpha = 0.05$), hence we can say table is significant as a predictor.

(f) Find the average estimated price of a diamond with an average table value and which is of fair cut.

```
-6563.672+179.105*mean(table)+365.568
```

```
## [1] 4092.765
```

The average estimated price is \$4092.765

Problem Set - 5

1. Problem to demonstrate the utility of K nearest neighbour regression over least squares regression.

Consider a setting with $n = 1000$ observations. Generate: * (i) x_{1i} from $N(0, 2^2)$ and x_{2i} from $Poisson(\lambda = 1.5)$. * (ii) ε_i from $N(0, 1)$. * (iii) $y_i = -2 + 1.4x_{1i} - 2.6x_{2i} + \varepsilon_i$

```
rm(list=ls())
set.seed(123)
n=1000
x1 = rnorm(n,0,2)
x2 = rpois(n,1.5)
e = rnorm(n)
y = -2 + 1.4*x1 - 2.6*x2 + e

data = data.frame(y,x1,x2)
head(data)

##           y          x1 x2
## 1 -4.3903185 -1.1209513  0
## 2 -2.9517542 -0.4603550  0
## 3  1.4622853  3.1174166  0
## 4 -3.7755078  0.1410168  1
## 5 -3.1176393  0.2585755  1
## 6 -0.2706045  3.4301300  2
```

Split the data into train and test sets. Keep the first 800 observations as training data and the remaining as test data. Work out the following:

```
train_data = data[1:800,]
test_data  = data[-(1:800),]
```

1. Fit a multiple linear regression equation of y on x_1 and x_2 . Calculate test MSE.

```
m1 = lm(y~x1+x2, data=train_data)
summary(m1)

##
## Call:
## lm(formula = y ~ x1 + x2, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.0727 -0.6573 -0.0125  0.6921  3.2412
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.07300    0.05382  -38.52  <2e-16 ***
## x1           1.38207    0.01767   78.21  <2e-16 ***
## x2          -2.55584    0.02768  -92.34  <2e-16 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.98 on 797 degrees of freedom
## Multiple R-squared:  0.9492, Adjusted R-squared:  0.9491
## F-statistic: 7445 on 2 and 797 DF,  p-value: < 2.2e-16

y_test = predict(m1,newdata = test_data)

test_mse = mean((y_test-test_data[,1])^2)
test_mse

## [1] 0.998901
```

2. Fit a KNN model with $k = 1, 2, 5, 9, 15$. Calculate test MSE for each choice of k .

```
library(ggplot2)
library(lattice)
library(caret)

mse_knn = c()
k_choices = c(1,2,5,9,15)
for(i in 1:5)
{
  kr = knnreg(y~x1+x2, data = train_data, k= k_choices[i])
  y_knn = predict(kr,test_data)
  mse_knn[i] = mean((test_data$y - y_knn)^2)
}

output = data.frame(k_choices,mse_knn,rep(test_mse,each=5))
colnames(output) = c("k Choices","KNN Test MSE","Multiple Linear Reg Test MSE")
output

##   k Choices KNN Test MSE Multiple Linear Reg Test MSE
## 1         1      2.219793                0.998901
## 2         2      1.729587                0.998901
## 3         5      1.303978                0.998901
## 4         9      1.205371                0.998901
## 5        15      1.232730                0.998901
```

We consider the function: $y_i = \frac{1}{(-2+1.4x_{1i}-2.6x_{2i}+2.9x_{1i}^2)} + 3.1\sin x_{2i} - 1.5x_{1i}x_{2i}^2 + \varepsilon_i$

Work out the problems in (1) and (2). Compare and comment on the results.

```
y1 = 1/(-2 + 1.4*x1 - 2.6*x2 + 2.9*(x1^2)) + 3.1*sin(x2) - 1.5*x1*(x2^2) + e

data1 = data.frame(y1,x1,x2)
head(data1)
```

```

##           y1           x1 x2
## 1  12.581968 -1.1209513  0
## 2   -0.799890 -0.4603550  0
## 3   -0.869362  3.1174166  0
## 4    2.793949  0.1410168  1
## 5    3.093778  0.2585755  1
## 6  -15.603221  3.4301300  2

train_data1 = data1[1:800,]
test_data1  = data1[-(1:800),]

m2 = lm(y1 ~x1+x2, data=train_data1)
summary(m2)

##
## Call:
## lm(formula = y1 ~ x1 + x2, data = train_data1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -241.819   -5.150   -0.051    5.566   155.662
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -0.2369     1.0272  -0.231  0.81764
## x1           -6.3747     0.3373 -18.901 < 2e-16 ***
## x2            1.3849     0.5283   2.621  0.00892 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 18.7 on 797 degrees of freedom
## Multiple R-squared:  0.3146, Adjusted R-squared:  0.3129
## F-statistic: 182.9 on 2 and 797 DF,  p-value: < 2.2e-16

y1_test = predict(m2,newdata = test_data1)

test_mse1 = mean((y1_test-test_data1[,1])^2)
test_mse1

## [1] 205.1776

mse_knn1 = c()
k_choices = c(1,2,5,9,15)
for(i in 1:5)
{
  kr1 = knnreg(y1 ~ x1+x2, data = train_data1, k= k_choices[i])
  y1_knn = predict(kr1,test_data1)
  mse_knn1[i] = mean((test_data1[,1] - y1_knn)^2)
}

output = data.frame(k_choices,mse_knn1,rep(test_mse1,each=5))

```

```
colnames(output) = c("k Choices", "KNN Test MSE", "Multiple Linear Reg Test MSE")
```

```
output
```

##	k Choices	KNN Test MSE	Multiple Linear Reg Test MSE
## 1	1	47.52490	205.1776
## 2	2	54.48942	205.1776
## 3	5	59.77963	205.1776
## 4	9	62.10913	205.1776
## 5	15	63.72303	205.1776

- From the above comparison we see that, KNN method performs much better than multiple linear regression. KNN produces a much lower test mean square error as the model is non-linear, hence multiple linear regression performs poorly.
- We observe that KNN model for $k = 1$ is minimum, but we see that the increment in MSE for different values of k is substantially small hence we can opt for a moderately high to high value of k .